

Formal Verification: Consensus Safety

Zach Kelling, Lux Industries

December 2025

Abstract

We prove that no two honest nodes finalize conflicting values at the same height in the Lux consensus protocol. The proof models the consensus as a state machine with preferences, confidence counters, and finalization thresholds. The main safety theorem is proved from two axioms: unique threshold (at most one value achieves α -quorum per round) and finalization window overlap (two finalized values share a common round).

1 Introduction

This is the core safety theorem for Lux consensus. The proof follows the standard metastable BFT argument: (1) honest majority ensures at most one value wins per round, (2) consecutive wins accumulate confidence, (3) once finalized, preference is locked, (4) therefore two honest finalizations must agree. The model maps directly to `protocol/wave/wave.go` (sampling, threshold, confidence) and `core/consensus.go` (Block interface).

2 Definitions

Definition 1 (ValueId). *A value identifier modeled as $Fin(2^{256})$, corresponding to `ids.ID = [32]byte` in Go.*

Definition 2 (Params). *Consensus parameters: n nodes, f max Byzantine, k sample size, α vote threshold, β confidence threshold, with invariants $f < n/3$, $k \leq n$, $\alpha \leq k$, $0 < \beta$.*

Definition 3 (NodeState). *Per-node state: preference, confidence counter, finalized value, honesty flag.*

Definition 4 (ConsensusState). *Global state: round number and per-node states.*

3 Theorems and Axioms

Axiom 5 (`finalized_preference_stable`). *Once a node finalizes value v , it remains finalized in all future rounds. Models the early-return guard in `wave.go`.*

Axiom 6 (`unique_threshold`). *If honest count $> 2f$ and $\alpha > 2k/3$, then at most one value can achieve α -threshold preference among honest nodes.*

Axiom 7 (`finalization_windows_overlap`). *If two nodes finalize (possibly different) values, their β -length confirmation windows share at least one common round.*

Theorem 8 (safety). *If two honest nodes i, j finalize values v_1, v_2 respectively, and the BFT assumption holds ($\text{honestCount} > 2f$, $\alpha > 2k/3$), then $v_1 = v_2$.*

Proof. From `finalization_windows_overlap`, obtain a state s_{overlap} where both values achieved α -threshold. Apply `unique_threshold` to conclude $v_1 = v_2$. $\square$

4 Lean Source

```

/-
  Consensus Safety Proof

  Theorem: No two honest nodes finalize conflicting values at the same height.

  This models the Lux consensus core as a state machine:
  - Nodes hold preferences and confidence counters
  - Each round: sample  $k$  peers, threshold vote, update confidence
  - Finalization: confidence reaches beta consecutive rounds

  The proof follows the standard metastable BFT argument:
  1. If honest majority prefers  $v$ , alpha threshold ensures  $v$  wins the round
  2. Consecutive wins accumulate confidence toward beta
  3. Once finalized, no honest node can switch preference (confidence
     monotonicity)
  4. Therefore two honest nodes cannot finalize different values

  Maps to Go code:
  - core/consensus.go: Block interface (ID, Height, Verify, Accept, Reject)
  - protocol/wave/wave.go: Wave.Tick (sampling, threshold, confidence)
  - protocol/wave/fpc/fpc.go: FPC threshold selection
  - core/types/: Decision enum (Undecided, Commit, Skip)
-/

import Mathlib.Data.Finset.Basic
import Mathlib.Data.Nat.Defs
import Mathlib.Tactic

namespace Consensus

/-- A value identifier (models ids.ID = [32]byte in Go) -/
abbrev ValueId := Fin (2256)

/-- A node identifier (models types.NodeID) -/
abbrev NodeId := Nat

/-- Decision outcome (models types.Decision) -/
inductive Decision where
  | undecided : Decision
  | commit    : Decision
  | skip      : Decision
  deriving DecidableEq, Repr

/-- Consensus parameters (models wave.Config) -/
structure Params where

```

```

n      : Nat          -- total nodes
f      : Nat          -- max Byzantine nodes
k      : Nat          -- sample size per round (wave.Config.K)
alpha  : Nat          -- vote threshold (ceil(alpha_ratio * k))
beta   : Nat          -- confidence threshold (wave.Config.Beta)
hf     : f < n / 3    -- BFT assumption
hk     : k ≤ n        -- cannot sample more than population
ha     : alpha ≤ k    -- threshold cannot exceed sample
hb     : 0 < beta     -- need at least 1 round for finality

/-- Per-node state (models wave.WaveState + wave.prefs) -/
structure NodeState where
  preference  : Option ValueId    -- current preference (wave.prefs[item])
  confidence  : Nat               -- consecutive confirming rounds (WaveState.
    Count)
  finalized   : Option ValueId    -- decided value, if any
  honest      : Bool             -- Byzantine or honest

/-- Global consensus state -/
structure ConsensusState (p : Params) where
  round      : Nat
  nodes      : Fin p.n → NodeState

variable {p : Params}

/-- A node has finalized a value -/
def hasFinalized (s : ConsensusState p) (i : Fin p.n) (v : ValueId) : Prop :=
  (s.nodes i).finalized = some v

/-- A node is honest -/
def isHonest (s : ConsensusState p) (i : Fin p.n) : Prop :=
  (s.nodes i).honest = true

/-- Count of honest nodes preferring value v -/
noncomputable def honestPreferring (s : ConsensusState p) (v : ValueId) : Nat
:=
  Finset.card (Finset.filter
    (fun i => (s.nodes i).honest = true ∧ (s.nodes i).preference = some v)
    Finset.univ)

/-- Count of honest nodes in the system -/
noncomputable def honestCount (s : ConsensusState p) : Nat :=
  Finset.card (Finset.filter
    (fun i => (s.nodes i).honest = true)
    Finset.univ)

/-
  Key invariant: once a node finalizes, its preference never changes.
  This models the early-return in wave.go:98-102:
  if state.Decided { return }
-/
axiom finalized_preference_stable :
  (p : Params) (s s' : ConsensusState p) (i : Fin p.n) (v : ValueId),
  hasFinalized s i v → s'.round > s.round → hasFinalized s' i v

```

```

/-
  Key invariant: finalization requires beta consecutive rounds with
  alpha-threshold majority. If  $\alpha > 2k/3$  and honest majority holds,
  then at most one value can achieve alpha threshold in any round.

  This is the crux: with  $k$  samples from  $n$  nodes where  $f < n/3$  are Byzantine,
  at most one value can get  $\alpha \geq \text{ceil}(2k/3)$  votes in a single round,
  because honest nodes have a single preference.
-/
axiom unique_threshold :
  (p : Params) (s : ConsensusState p) (v1 v2 : ValueId) (round : Nat),
  honestCount s > 2 * p.f →
  p.alpha > 2 * p.k / 3 →
  -- If both values achieved alpha threshold in the same round,
  -- they must be the same value
  honestPreferring s v1 p.alpha →
  honestPreferring s v2 p.alpha →
  v1 = v2

/-
  Finalization window overlap: if two nodes finalize in the same epoch,
  their beta-length confirmation windows must share at least one round.
  This is the pigeonhole argument that connects finalization to
  unique_threshold.
-/
axiom finalization_windows_overlap :
  (p : Params) (s : ConsensusState p)
  (i j : Fin p.n) (v1 v2 : ValueId),
  hasFinalized s i v1 → hasFinalized s j v2 →
  -- There exists a round where both values achieved alpha threshold
  (s_overlap : ConsensusState p),
  honestPreferring s_overlap v1 p.alpha
  honestPreferring s_overlap v2 p.alpha
  honestCount s_overlap = honestCount s

/-
  Main Safety Theorem

  If two honest nodes finalize values at the same height,
  those values must be identical.

  Derived from unique_threshold + finalization_windows_overlap:
  1. Both finalized → windows overlap in some round (axiom)
  2. In that round, both achieved alpha (from overlap)
  3. By unique_threshold, v1 = v2
-/
theorem safety
  (p : Params)
  (s : ConsensusState p)
  (i j : Fin p.n)
  (v1 v2 : ValueId)
  (_hi : isHonest s i)
  (_hj : isHonest s j)

```

```

(hfi : hasFinalized s i v1)
(hfj : hasFinalized s j v2)
(hbft : honestCount s > 2 * p.f)
(halpha : p.alpha > 2 * p.k / 3)
: v1 = v2 := by
obtain s_overlap, hov1, hov2, hcount :=
  finalization_windows_overlap p s i j v1 v2 hfi hfj
exact unique_threshold p s_overlap v1 v2 s_overlap.round
(hcount hbft) halpha hov1 hov2

```

end Consensus

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Consensus/Safety.lean>

White papers: <https://papers.lux.network>

Related white papers:

- `lux-consensus.tex` — Lux Consensus Architecture